

GetGuac — Cross-Platform Test Report

Generated 2026-06-01 · 31 fixture cases × 2 engines (Web JS + Mobile Dart)

62 / 62

TOTAL ASSERTIONS PASSING

31

FIXTURE CASES

2

ENGINES (JS + DART)

3

BUGS CAUGHT + FIXED

What this report verifies

Both engines — `web/src/lib/wizardScore.js` + `web/src/lib/financeInsights.js` on the JS side, and `mobile/lib/services/wizard_score.dart` on the Dart side — run the SAME input fixture and produce identical output. If they drift, the dashboard tile and the `/guacanomics /` `/guacwizard` pages would disagree for the same user. The fixture file at `test-fixtures/score-engines.json` is the contract.

How to re-run

```
cd web    && node scripts/test-score-engines.mjs      (JS engine)
cd mobile && flutter test test/services/wizard_score_test.dart (Dart engine)
cd web    && npm run test:parity                       (both)
```

1. computeWizardScore — score-formula tests (23 cases)

Pure function: `summary` + `accounts` → score 0-100.

#	Case	Inputs	Expected	Web	Mobile	Status
1	no_data_baseline	no accounts, no totals	50	50	50	✓ pass
2	clean_user_no_costs	0 fees, payments=purchases	100	100	100	✓ pass
3	interest_50_dollars	\$50 interest	95	95	95	✓ pass
4	high_apr_card_penalty	APR 26.99%	95	95	95	✓ pass
5	interest_350_capped	\$500 interest → cap -35	65	65	65	✓ pass
6	debt_growing	netDebtChange=+\$400	92	92	92	✓ pass
7	debt_shrinking_bonus	netDebtChange=-\$500	100	100	100	✓ pass
8	fees_paid_25_dollars	\$25 fees	95	95	95	✓ pass
9	two_high_apr_cards	2 cards ≥ 25% APR	90	90	90	✓ pass
10	extreme_high_interest_capped	\$10000 interest	65	65	65	✓ pass

11	extreme_fees_capped_at_20	\$1000 fees	80	80	80	✓ pass
12	high_apr_AND_high_interest	stacking	75	75	75	✓ pass
13	all_metrics_in_play	4 penalties stacking	66	66	66	✓ pass
14	apr_just_below_threshold	24.99%	100	100	100	✓ pass
15	apr_exactly_at_threshold_25	25%	95	95	95	✓ pass
16	tiny_interest_rounds_zero	\$0.01 interest	100	100	100	✓ pass
17	five_high_apr_cards_stacks	5 × -5	75	75	75	✓ pass
18	null_apr_accounts	2 accounts, null APR	100	100	100	✓ pass
19	fractional_fees	\$27.50 fees	94	94	94	✓ pass
20	debt_growing_capped_at_20	\$10K purchases	80	80	80	✓ pass
21	huge_debt_shrink_capped_at_10	bonus +10 max	100	100	100	✓ pass
22	refunds_pull_debt_negative	refunds > payments	100	100	100	✓ pass
23	totalPurch_legacy_field	old field name accepted	92	92	92	✓ pass

2. aggregateBankForWizard — pipeline tests (8 cases)

Raw bank rows → summary + accounts. This layer was historically the source of most cross-platform drift.

#	Case	Setup	Expected interest / fees / accts	Web	Mobile	Status
24	fees_only_no_statements	Fees but no statements	60 / 30 / 1	60 / 30 / 1	60 / 30 / 1	✓ pass
25	everything_empty	All arrays empty	0 / 0 / 0	0 / 0 / 0	0 / 0 / 0	✓ pass
26	fees_outside_window	Date before startStr	0 / 0 / 0	0 / 0 / 0	0 / 0 / 0	✓ pass
27	penalty_classified_as_fee	kind=penalty	0 / 35 / 1	0 / 35 / 1	0 / 35 / 1	✓ pass
28	transactions_with_flags	is_interest, is_fee, is_payment	25 / 15 / 1	25 / 15 / 1	25 / 15 / 1	✓ pass
29	boundary_date_at_start_included	Date == startStr	50 / 0 / 1	50 / 0 / 1	50 / 0 / 1	✓ pass
30	boundary_date_at_end_excluded	Date == endStr (exclusive)	0 / 0 / 0	0 / 0 / 0	0 / 0 / 0	✓ pass

31	mixed_fees_and_txns_no_dedup	Same incident in both tables	120 / 0 / 1	120 / 0 / 1	120 / 0 / 1	✓ pass
----	------------------------------	------------------------------	-------------	-------------	-------------	-----------

3. Bugs caught + fixed during this test run

The whole reason for fixture testing. Each of these would have caused real user-facing drift between web and mobile.

Bug 1 — Web's `bankAccountTotals` ignored end-date bound

Layer	Symptom	Root cause	Fix
JS <code>financeInsights.inRange()</code>	Row dated == endStr was INCLUDED on web but EXCLUDED on Dart	JS only checked <code>since</code> (single Date). No upper-bound support.	Extended <code>inRange</code> to accept <code>{ start, end }</code> object with exclusive upper bound. <code>getPeriodStart</code> passes through the new shape.

Bug 2 — Wrong expected values in fixtures (writer error)

Fixture	Was	Should be	Math
<code>all_metrics_in_play</code>	56	66	100 - 8 (interest) - 8 (fees) - 13 (debt) - 5 (APR) = 66
<code>refunds_pull_debt_negative</code>	105	100	Bonus +5 clamps at final <code>min(100, ...)</code>

4. Cross-platform drift bugs caught in earlier sessions

#	Bug	Status
1	JS used <code>totalPurch</code> , Dart used <code>totalPurchases</code> — same value, different name	Fixed; engine accepts both
2	JS expected <code>summary.netDebtChange</code> pre-computed, Dart derived it	Fixed; engine falls back to computing from raw fields
3	Both engines tripped baseline-50 when fees existed but statements were empty	Fixed; both engines now synthesize a placeholder account
4	Supabase queries non-deterministic order → web vs mobile picked different "latest" statement	Fixed; explicit <code>.order('period_end', desc).order('id')</code> everywhere
5	<code>periodStartDate</code> returned ms-precision Date → boundary day flipped in/out across renders	Fixed; snap to midnight local
6	Detail screens called <code>loadReceipts()</code> with default month period → shrinking the dashboard's all-history cache	Fixed; <code>ReceiptPeriod.all</code> on every detail screen

5. Test infrastructure summary

File	Purpose
------	---------

test-fixtures/score-engines.json	31 input/output pairs, language-neutral JSON
web/scripts/test-score-engines.mjs	JS runner — Node, no test framework, plain assertions
mobile/test/services/wizard_score_test.dart	Dart runner — flutter_test, reads same JSON
mobile/test/navigation_back_button_audit_test.dart	Static-source audit: every Scaffold has AppBar; main.dart wires Cupertino transitions; dashboard uses context.push()
npm run test:parity	Runs both sides end-to-end

6. What's NOT covered yet (honesty section)

- **Widget rendering / layout** — automated visual diffs would need golden-file tests. Manual eyeball + screenshots cover this today.
- **Real-data scenarios** — fixtures are synthetic. Your actual receipts/bank rows exercise paths fixtures don't.
- **End-to-end integration** — no test pumps the dashboard, simulates a tap, asserts the destination screen rendered. Possible to add but each test takes 10-30s to run.
- **analysisEngine.computeDashboardAnalysis** — has its own 8-metric output but no parity fixtures yet. (Each tile's value/delta is computed by it.)
- **GuacScore engine** (`calculateGuacoScore`) — uses a different formula than WizardScore; fixtures not yet written.

7. To convert this report to PDF

Open this file in any browser (double-click `test-fixtures/TEST_REPORT.html`) → **Ctrl+P** → Destination: **Save as PDF** → Save. Layout is print-styled (A4, narrow margins).